PROJECT REPORT | OCTOBER 2026

EDUCATIONAL RESEARCH PROTOTYPE

Liver Tumor Segmentation and Analysis

A 2D U-Net workflow for CT volume segmentation, measurement, and review

This report describes the software, data pipeline, model, user interface, current experiment artifacts, and limitations of the liver tumor segmentation project.

Project type	Educational medical-imaging research prototype
Input	3D NIfTI CT volume
Output	Voxel labels for background, liver, and tumor, with visual overlays and image-derived measurements
Implementation	Python, PyTorch, NiBabel, NumPy, Streamlit, and PyMuPDF

Important: This prototype is not clinically validated, is not a medical device, and must not be used to diagnose disease or make treatment decisions. All outputs require qualified clinical review.

Prepared from the project source, configuration, available data and model artifacts, and the test suite present in the workspace on 23 October 2026.

1. Executive summary

The project implements an end-to-end research workflow for semantic segmentation of liver and tumor regions in abdominal CT. Its core model is a compact 2D U-Net with three output classes. Training uses annotated NIfTI volumes, slice-wise preprocessing, a patient/volume-level split, and a combined cross-entropy plus Dice loss. The Streamlit application provides a workflow for entering a patient record, uploading a NIfTI scan, reviewing slices and overlays, inspecting measurements, and exporting a text report and segmentation mask.

> **Current evidence:** Four paired image/label volumes, a model checkpoint, and a two-epoch training history are present in the workspace. The recorded validation loss increased from 1.8458 at epoch 1 to 3.0185 at epoch 2. No held-out test-metric output is present, so this report makes no claim about Dice, IoU, or clinical performance.

Problem and motivation

Manual contouring of liver and tumor regions in 3D CT can be time-consuming. This prototype explores a reproducible segmentation pipeline and makes model-generated regions easier to inspect and quantify. It is a technical demonstration, not a replacement for radiologists or validated clinical software.

Aim and objectives

- Load 3D NIfTI CT and label volumes while retaining spatial metadata.
- Normalize image intensities and validate expected label IDs.
- Split data by volume/patient to reduce slice-level leakage.
- Train a 2D U-Net to predict background, liver, and tumor classes.
- Evaluate liver and tumor separately with segmentation metrics.
- Visualize predictions and calculate physical volumes only when spacing and units are valid.

Project boundary

The current pipeline accepts NIfTI volumes; direct DICOM-series ingestion is not implemented. Predictions and derived measurements are model outputs and must not be interpreted as a diagnosis.

2. System design and methodology

Processing pipeline

Stage	Implementation
Data loading	NiBabel loads NIfTI images and labels. The inference path preserves the NIfTI image object and uses its affine/header when saving a mask.
Validation	Image and label dimensions must match and be 3D. Label IDs are restricted to 0, 1, and 2.
Preprocessing	CT intensities are clipped to [-100, 400] HU and scaled to [0, 1]. Images are padded to multiples of 16 for the encoder/decoder.
Splitting	Matching image-label volumes are shuffled with a fixed seed. The checked-in split file uses seed 42 and separates train, validation, and test volumes.
Training	PyTorch 2D U-Net; AdamW optimizer; cross-entropy plus foreground Dice loss; best checkpoint chosen by validation loss.
Inference	Axial slices are processed in batches; class predictions are restored to original in-plane dimensions.
Review/export	Streamlit displays CT slices and segmentation overlays, calculates voxel/volume measures, and exports a report and NIfTI mask.

Model architecture

The U-Net uses four encoder levels, a bottleneck, and four decoder levels. Each level uses two 3x3 convolutions with batch normalization and ReLU. Max pooling downsamples features; transposed convolutions upsample; skip connections concatenate corresponding encoder features with decoder features. A 1x1 convolution produces three class logits per pixel.

Training objective

Training combines multi-class cross-entropy with a soft Dice loss calculated over foreground classes (liver and tumor). AdamW is configured with a learning rate of 0.001 and weight decay of 0.0001; the default seed is 42. Training is configured for 30 epochs by default, but the available history records only two epochs.

> **Class encoding:** 0 = background; 1 = liver; 2 = tumor. Dataset label conventions should be verified before training.

3. Data, evaluation, and observed status

Workspace data and split

The workspace contains four matching NIfTI image/label pairs (volume-000 through volume-003). The split file assigns two volumes to training, one to validation, and one to testing. This is a very small dataset for model development and cannot establish generalizable performance.

Training record available

Epoch	Training loss	Validation loss
1	1.6526	1.8458
2	1.2618	3.0185

The validation loss rose between the two recorded epochs. This small log is not sufficient to characterize convergence or select a reliable model. A best-model checkpoint is present, but the held-out evaluation command has not produced a test metrics file in the project outputs directory.

Evaluation metrics implemented

The evaluation code computes class-specific Dice, IoU, precision, sensitivity/recall, and specificity for liver and tumor on each test volume, then exports per-patient and mean summaries. Dice and IoU measure overlap; precision and sensitivity characterize false-positive and false-negative behavior. Specificity may appear high when background dominates. HD95 is not implemented.

> **Results not reported:** No held-out liver/tumor metric values are available in the workspace at report time. Training loss is not a substitute for test-set segmentation performance. Do not infer or fabricate test results from the existence of a checkpoint.

Validation performed for this report

The project test suite completed successfully: **3 tests passed**. The tests cover metric behavior (perfect overlap and no overlap) and batched volume inference with progress reporting. Passing unit tests verifies selected code paths; it does not validate model accuracy or clinical safety.

Observed environment: Python 3.10; PyTorch 2.13.0+cpu; NumPy 1.26.4; pandas 2.3.3; NiBabel 5.4.2; Streamlit 1.58.0; PyMuPDF 1.28.2.

4. Application and measurements

Streamlit workflow

- Collects patient name, record number, and patient ID before entering the workspace.
- Accepts a 3D NIfTI CT volume (.nii or .nii.gz) and optionally a PDF report for preview.
- Shows slice-by-slice inference progress and reuses a completed result for the same scan during the session.
- Provides CT Review, AI-Assisted, Segmentation, Treatment Discussion, and Clinical Reporting sections.
- Displays CT slices with adjustable window/level, liver/tumor overlays, voxel counts, and segmentation summaries.
- Exports a text research summary and predicted segmentation as NIfTI; an uploaded PDF can be previewed and downloaded.

Image-derived measurements

The application counts predicted tumor voxels and liver-plus-tumor voxels. It converts voxel counts to cubic centimeters only when all three voxel spacings are finite and positive and the NIfTI spatial unit explicitly reports millimeters. Otherwise, it explains why physical volume is unavailable. These quantities describe model output; they are not validated clinical measurements.

Input and operational limits

- Only 3D NIfTI input is supported for CT inference; DICOM conversion is outside the application.
- Successful inference requires a trained checkpoint at `checkpoints/best_model.pt`.
- Inference depends on matching preprocessing, label conventions, and spatial metadata.
- The current inspected PyTorch installation is CPU-only; inference speed depends on local hardware and scan size.

> **Clinical safety:** The treatment discussion presents general categories and questions for a qualified care team; it does not recommend patient-specific treatment. Segmentation outputs must not guide treatment.

5. Reproduction guide and next steps

Windows setup

1. Use Python 3.10 or 3.11 and create/activate a virtual environment.
2. Install a compatible PyTorch build using the official PyTorch selector, then install dependencies from `requirements.txt`.
3. Place the dataset in `data/Task03_Liver/imagesTr` and `data/Task03_Liver/labelsTr`; verify labels are background 0, liver 1, tumor 2.
4. Generate patient/volume splits, train the model, then run held-out evaluation before describing performance.
5. Launch the interface with `streamlit run app.py`.

Commands

```
python -m src.make_splits --images data/Task03_Liver/imagesTr --labels
data/Task03_Liver/labelsTr --out data/splits.json
python -m src.train --splits data/splits.json --epochs 30 --batch-size 4
python -m src.evaluate --splits data/splits.json --checkpoint checkpoints/
best_model.pt
streamlit run app.py
```

Recommended next steps

- Verify dataset provenance, license, label IDs, patient identity handling, and train/validation/test disjointness.
- Train with a sufficiently large, representative dataset; monitor validation behavior and record settings and software versions.
- Run held-out evaluation and report per-patient, per-class results with dispersion; inspect false positives and false negatives.
- Stratify tumor results by lesion size and include empty/small-lesion cases; consider external validation and HD95.
- Review output NIfTI orientation, affine, units, and measurements with domain experts before any research publication or deployment.

Selected references and project documentation

- Medical Segmentation Decathlon, Task03 Liver: https://medicaldecathlon.com/
- Simpson et al., ?A large annotated medical image dataset for the development and evaluation of segmentation algorithms,? https://arxiv.org/abs/1902.09063
- Bilic et al., ?The Liver Tumor Segmentation Benchmark (LiTS),? Medical Image Analysis (2023), https://doi.org/10.1016/j.media.2022.102680
- Isensee et al., ?nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation,? Nature Methods (2021), https://doi.org/10.1038/s41592-020-01008-z

This report summarizes the implementation and artifacts available in the project workspace. It is not a clinical validation report.